



Introducción a Prompt Engineering

En **Sprint 4** aprendiste a **conectar** con un modelo de lenguaje desde Python: API key, cliente Gemini, primera llamada, a veces JSON.

En **Unidad 1** de este sprint aprendiste a **organizar** el código: validar datos, controlar el flujo, separar **config**, **prompts**, **logic**, **state** y **main**.

En **esta unidad** el foco cambia:

Ya sabemos llamar a un LLM. Ahora vamos a aprender a controlarlo.

No se trata de “escribir mejor en general”. Se trata de **diseñar instrucciones** que el modelo siga con más predecibilidad, **construirlas desde Python** y **tratar la respuesta como dato** que tu programa puede usar.

Objetivos de la unidad

Al terminar, deberías poder:

- **Diseñar prompts más eficaces** (rol, tarea, restricciones, formato).
- **Modificar el comportamiento del modelo** sin cambiar de modelo ni de API.
- **Construir prompts dinámicamente** con variables, plantillas y funciones.
- **Reutilizar** plantillas y patrones de llamada.
- **Incorporar datos externos** (usuario, configuración, ficheros, dicts) en las instrucciones.
- **Pedir y procesar salidas estructuradas** (listas, JSON) para integrarlas en código.

Qué es (y qué no es) Prompt Engineering

Sí es	No es
Definir rol , tarea , restricciones y formato de salida	Elegir el “mejor modelo del mercado”
Parametrizar instrucciones con Python	Pegar prompts largos en un solo print y olvidarlos
Separar datos / prompt / llamada / parseo	Magia: “si el prompt es bonito, siempre funciona”
Iterar y comparar resultados	Una sola frase suelta sin contexto ni criterio

Prompt Engineering aquí significa: **ingeniería de la interfaz entre tu programa y el LLM** — sobre todo el **texto que envías** y el **texto que recibes**.

Prerrequisitos (lo que ya deberías tener)

- Python 3.10+ y uso básico de funciones, `dict`, f-strings.
- Unidad 1: validación, respuestas `ok/error`, idea de `prompts.py`.
- Sprint 4: `google-genai`, variable `GEMINI_API_KEY`. En notebooks y proyectos: `load_dotenv()` y, si falta la clave, `getpass` (entrada oculta, solo esa sesión):

```
import os
import getpass

from dotenv import load_dotenv
from google import genai

load_dotenv()

if not os.getenv("GEMINI_API_KEY"):
    os.environ["GEMINI_API_KEY"] = getpass.getpass(
        "Pega aquí tu GEMINI_API_KEY (input oculto): "
    )

print("GEMINI_API_KEY configurada:", "sí" if os.getenv("GEMINI_API_KEY") else "no")

client = genai.Client()
response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="...",
)
print(response.text)
```

En los ejemplos de esta unidad usamos **Gemini** de forma consistente; el **patrón** (construir prompt → llamar → leer salida) sirve para otros proveedores.

Hilo narrativo: tickets de tarea (continuidad con Unidad 1)

En Unidad 1 trabajaste con registros del estilo:

```
{
    "nombre": "Ana",
    "codigo": "TKT-1234",
    "tipo": "summary",      # summary | translate | qa
    "prioridad": 3,
}
```

Aquí reutilizamos esa idea: el LLM puede **resumir**, **traducir**, **responder preguntas** o **clasificar** incidencias según cómo construyas el prompt — no porque el modelo “sepa” tu negocio, sino porque **tú** le das rol, tarea y datos.

Así la unidad se siente continua: primero estructuras el código; ahora estructuras lo que le dices al modelo.

Mapa de contenidos (orden de lectura)

#	Documento	Tema central
00	Este archivo	Panorama, objetivos, convenciones
01	Rol del modelo	Quién “actúa” el modelo: tono, audiencia, expertise
02	Definición de tareas	Qué debe hacer: especificidad, entregables, restricciones
03	Prompts dinámicos	Plantillas, variables, funciones <code>build_prompt</code>
04	Inputs de usuario	Parámetros que el usuario o la app aportan al prompt
05	Inyección de datos externos	CSV, APIs, dicts: información de referencia en el prompt
06	Salidas estructuradas	JSON, listas, validación y parseo en Python

La pieza que une todo: `build_prompt`

Al final de la unidad, la mentalidad objetivo es esta:

```
prompt = build_prompt(
    role=role,
    task=task,
    user_input=user_input,
    additional_data=additional_data,
)
```

- **role** — identidad y estilo (doc 01).
- **task** — instrucciones concretas y entregables (doc 02).
- **user_input** — texto o parámetros del usuario (docs 03–04).
- **additional_data** — datos de ficheros, APIs, sesión (doc 05).
- **Formato de salida** — pedido en la tarea y reforzado en la llamada (doc 06).

Tu programa:

1. Valida inputs (herencia Unidad 1).
2. Construye el string `prompt` (esta unidad).
3. Llama a Gemini.
4. Interpreta la respuesta — sobre todo si pediste JSON (doc 06 + Unidad 1).

Convenciones en los ejemplos

API y modelo

- Clave: `.env` con `load_dotenv()` o `getpass` si no está definida (como en Sprint 4).
- Cliente: `genai.Client()` (lee `GEMINI_API_KEY` del entorno).
- Modelo de ejemplo: `gemini-3-flash-preview` (ajusta al que use tu campus si cambia).
- Temperatura baja (`0` o `0.2`) cuando busques **formato estable** (JSON, clasificación).

Organización del código (recomendada)

Alineado con Unidad 1:

```
proyecto/
├── config.py      # MODEL, temperatura, límites
├── prompts.py    # plantillas y build_prompt(...)
├── gemini_client.py # opcional: wrapper de llamada
├── logic.py      # validar, parsear respuesta
└── main.py      # orquestación
```

No es obligatorio en cada ejercicio de lectura; sí es el **destino** hacia el que evoluciona tu práctica.

Seguridad

- No subas API keys al repositorio.
- En ejemplos con datos de usuario o CSV, usa datos ficticios en clase.

Errores de principiante (para evitar desde el día 1)

1. **Confundir pregunta con instrucción** — “¿Qué es X?” sin rol ni formato vs “Actúa como... Explica X en 3 puntos...”.
2. **Prompt gigante sin estructura** — un párrafo; mejor secciones: ROL, TAREA, DATOS, FORMATO.
3. **Mezclar datos y lógica en un solo string sin plantilla** — difícil de mantener; usa f-strings o plantillas en `prompts.py`.
4. **Asumir que el modelo siempre obedece el JSON** — hay que **parsear y validar** (doc 06).
5. **Llamar “contexto” a todo** — en esta unidad hablamos de **datos en el prompt**; “Context Engineering” es la siguiente capa.

Cómo estudiar esta unidad

1. Lee en orden **00** → **06**.
2. Ejecuta los fragmentos Python (notebook local, Colab o script) con tu API key.
3. Compara respuestas cambiando **solo una variable** (rol, tarea, dato inyectado).
4. Anota qué cambió en la salida: eso es “controlar el modelo”.

Los **workouts** (cuando existan) serán la parte más limitada en tiempo de clase; la teoría está pensada para profundizar por tu cuenta.

Resumen en una frase

Prompt Engineering = escribir y generar desde Python las instrucciones que el LLM necesita, y diseñar la salida para que tu código pueda confiar en ella (con validación).